

we show the practical algorithm. Finally, in Section 4.4, we demonstrate our system co-design for accurate and efficient LLM decoding through GPU-CPU collaboration.

Note that most of the derivations in this section might be classical and can even be found in textbooks, but our goal is to leverage them to motivate MAGICPIG design and precisely demonstrate the power of a rigorously sound algorithm with system co-design in deep generative models.

## 4.1 Self-normalized importance sampling for attention estimation

Oracle sampling estimation cannot go beyond  $2\times$  wall clock speed up because obtaining distribution  $w$  requires full computation of all  $qk_i^T$ , thereby only saving the  $wV$  computation.

Fortunately, importance sampling (Kloek and Van Dijk, 1978; Owen, 2013; Lohr, 2021) allows us to estimate unknown distribution  $w$  by sampling from a proposed distribution  $u$ . In our problem setting, the normalization factor of  $w$ , i.e.  $Z = \sum_{i=1}^n \exp \frac{qk_i^T}{\sqrt{d}}$  is also unknown because computing it requires evaluating all  $qk_i^T$ . However,

we do have access to unnormalized weights  $\widetilde{w}_i = e^{\frac{qk_i^T}{\sqrt{d}}}$  for sampled indices  $i$ . Hence, by employing a variant of importance sampling, **self-normalized importance sampling** (Owen, 2013), we sample indices  $i_1, i_2, \dots, i_B$  from a proposed distribution  $u$  and the resulting estimator is

$$X^{\text{IS}} = \frac{1}{\widetilde{Z}} \sum_{j=1}^B \frac{\widetilde{w}_{i_j}}{u_{i_j}} v_{i_j} \quad \text{where} \quad \widetilde{Z} = \sum_{j=1}^B \frac{\widetilde{w}_{i_j}}{u_{i_j}} \quad (6)$$

which has a very nice property for accurately estimating attention output that  $\mathbb{P}[\lim_{B \rightarrow \infty} X^{\text{IS}} = o] = 1$ .

Its variance<sup>1</sup> is related to the distribution  $u$ , and can be approximated by

$$\text{Var}(X^{\text{IS}}) = \frac{1}{B} \mathbb{E}_{i \sim u} \left[ \frac{w_i^2}{u_i^2} (v_i - o)^2 \right] = \frac{1}{B Z^2} \mathbb{E}_{i \sim u} \left[ \frac{\widetilde{w}_i^2}{u_i^2} (v_i - o)^2 \right] \quad (7)$$

To minimize the variance,  $u$  should satisfy  $u \propto \widetilde{w}_i |v_i - o|$  (Hesterberg, 2003). The variance will be high if  $u_i$  and  $\widetilde{w}_i |v_i - o|$  assign a high probability mass to different regions of the sample space or have different modes. Therefore, the challenge is computing a distribution  $u$  aligned with  $\widetilde{w}_i |v_i - o|$  without accessing too many  $\widetilde{w}_i$ . Besides, Equation (6) requires that sampling probability  $u$  can be computed and  $u_i > 0$ , which is not satisfied by many deterministic approximations like TopK.

## 4.2 Variance reduction with LSH

We decompose  $\widetilde{w}_i |v_i - o| = \exp(\frac{qk_i^T}{\sqrt{d}} + \log |v_i - o|)$ . We observe empirically (Figure 10 in the appendix) that  $\log |v_i - o|$  does not fluctuate significantly compared to  $\frac{qk_i^T}{\sqrt{d}}$ . Hence, we simplify the requirement of  $u$  to share the same peaks with  $qk_i^T$ . By the following transformation,

$$r = \max_{1 \leq i \leq n} |k_i| \quad \bar{q} = [q, 0] \quad \bar{k}_i = [k_i, \sqrt{r^2 - |k_i|^2}] \quad (8)$$

we further transfer the inner product  $qk_i^T$  to cosine similarity between  $\bar{q}$  and  $\bar{k}_i$  (which is a common practice in Maximum Inner Product Search (Shrivastava and Li, 2014)).

Inspired by prior work (Spring and Shrivastava, 2017; Chen et al., 2020a), we leverage Locality sensitive hashing-based sampling for this estimation problem. Specifically, leveraging a hash function  $h$  in the LSH family that preserves cosine similarity such as SimHash (Sadowski, 2007), we can sample from probability distribution  $u_i = \mathbb{P}[h(q) = h(k_i)]$  which is monotonic to  $\cos \frac{qk_i^T}{|q| \cdot |k_i|}$ .

<sup>1</sup>We assume head dimension  $d = 1$  here for simplicity. Higher dimensions have similar formulations and analyses by replacing variance with the trace of covariance.

### 4.3 Algorithm Design

To make this estimation practical, MAGICPIG is implemented by the following specific design.

**Estimator approximation.** Self-normalized important sampling Equation (6) requires  $i_1, i_2, \dots, i_k$  iid sampled, but the probabilities provided by hashing are not normalized. Hence we adapt our estimator: After obtaining  $S$  with probability  $u$ , MAGICPIG computes

$$X = \frac{\sum_{i=1}^n \frac{\bar{w}_i}{u_i} v_i \mathbf{1}_{i \in S}}{\sum_{i=1}^n \frac{\bar{w}_i}{u_i} \mathbf{1}_{i \in S}} = \frac{\sum_{i \in S} \frac{\bar{w}_i}{u_i} v_i}{\sum_{i \in S} \frac{\bar{w}_i}{u_i}} \quad (9)$$

**Hash function selection.** MAGICPIG leverages **SimHash** (Sadowski, 2007), that draws with  $K \times L$  random vectors. For each of the  $L$  hash tables, the  $q$  and  $k_i$ s vectors are projected on  $K$  directions, and only the sign of the projection is kept, which yields a  $K$ -bit hash value. Key  $k_i$  is sampled only if there exist at least **two**<sup>2</sup> hash tables where  $k_i$  shares the hash value with  $q$ . The corresponding probability is

$$u_i = \mathbb{P}[k_i \text{ is sampled}] = 1 - (1 - p^K)^L - Lp^K(1 - p^K)^{L-1} \quad \text{where} \quad p = 1 - \frac{1}{\pi} \arccos \frac{qk_i^T}{|q| \cdot |k_i|} \quad (10)$$

**Data pre-processing.** Before building hash tables, MAGICPIG centers the  $k_i$  vectors. As shown in Figure 2c, keys are almost always concentrated on one side of the queries, except the initial token. In this case, random projections cannot effectively distinguish keys, resulting in uniform sampled probabilities. Softmax is translation invariant. Centering ( $\bar{k}_i = k_i - \frac{1}{n} \sum_{i=1}^n k_i$ ) distributed the keys better and remains computationally equivalent.

Combining Equations (9) and (10) gives a closed form of the MAGICPIG attention estimation. Assuming sample set  $S$  is obtained with LSH,

$$\begin{aligned} \bar{o} &= \sum_{i \in S} \frac{\exp(\frac{qk_i^T}{\sqrt{d}} - \log u_i)}{\sum_{i \in S} \exp(\frac{qk_i^T}{\sqrt{d}} - \log u_i)} v_i \\ u_i &= 1 - (1 - p^K)^L - Lp^K(1 - p^K)^{L-1} \\ p_i &= 1 - \frac{1}{\pi} \arccos \frac{qk_i^T}{|q| \cdot |k_i|} \end{aligned} \quad (11)$$

### 4.4 System co-design

The memory size of KV cache remains a bottleneck for long-context LLM decoding, especially when GPU VRAM is limited. DRAM on the CPU side offers sufficient memory capacity with 100 – 200GB/s bandwidth, which is usually 10 – 20% of GPU VRAM bandwidth (see Figure 7a). Ideally, this gap can be mitigated by 5 – 10× sparsity. To make CPU DRAM an *aggregated memory* for GPU, the workload must be partitioned. In our experiments,  $K = 9$  or 10, and  $L$  is a few hundred.

Our system design extends prior work (He and Zhai, 2024; Aminabadi et al., 2022) by splitting LLM decoding into three parts. (1) Parameter computations, i.e., all linear projectors including MLP and  $W_Q, W_K, W_V$ , and  $W_O$  in the self-attention module run on GPU. (2) Attention computation, which involves  $o = \text{Softmax}(\frac{qK^T}{\sqrt{d}})V$ , runs on CPU. (3) Random projections. At generation time, for each  $q$ ,  $K \times L$  random projections are conducted to obtain the hash codes. Since all heads can share the same random projectors, the memory overhead is limited (400 KB in our implementation), so this step is compute-bound. Therefore, the projection is placed on GPU.

---

#### Algorithm 1: MAGICPIG Decoding

---

**Input:**  $K, V \in R^{n \times d}$ ,  $q \in R^{1 \times d}$ , random projectors  $W \in R^{d \times (K \times L)}$ , hash tables  $HT$ , static KV cache  $K_T, V_T \in R^{t \times d}$ .

*Compute hash code for new query*

$q_{\text{code}} = \text{Encode}(q, W)$

*Query hash tables to sample  $S$  in Equation (9)*

$S = \text{Query}(HT, q_{\text{code}}), K_S = K[S], V_S = V[S]$

*Compute inner product for  $q$  and sampled  $K$*

$w_S = qK_S^T, w_T = qK_T^T$

*Compute collision probability for each hash function*

$p = 1 - \frac{1}{\pi} \arccos(w / (|q| \cdot ||K_S||))$

*Compute sampling probability*

$u = 1 - (1 - p^K)^L - Lp^K(1 - p^K)^{L-1}$

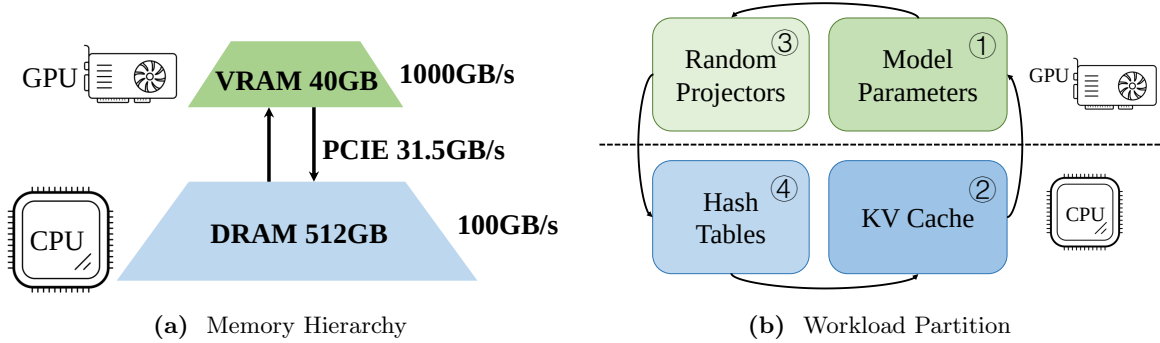
*Compute attention output estimation*

$\bar{o} = \text{Softmax}(\frac{[w_S, w_T]}{\sqrt{d}} - \log([u, 1_t]))[V_S, V_T]$

**Return**  $\bar{o}$

---

<sup>2</sup>Empirical results show that requiring hits in two hash tables greatly improves accuracy over standard SimHash.



**Figure 7 Left:** Memory hierarchy of hardware. GPU VRAM has high bandwidth but is limited. CPU DRAM is sufficient but is relatively slow. The limited bandwidth of PCIE forbids large-scale data transfer. **Right:** Workload partition of MAGICPIG. Linear projections and hash function computation (by random projection) are done on GPU, while sampling with hash tables and attention are done on CPU. The execution order is ①③④② at decoding time.

(4) Retrieval. The hash codes of  $q$ , need to be looked up in  $L$  hash tables, which is negligible computationally. However, the pre-built hash tables for  $k_i$ s can occupy considerable memory, making it a better fit for the CPU. With the above partition, we are able to support hash tables with  $K$  and  $L$  beyond the scale of prior work (Kitaev et al., 2020; Chen et al., 2021; Zandieh et al., 2023) without worrying about computation for hash codes as well as the storage of hash tables.

**On-device cache.** Sink tokens (the first several tokens) and local tokens are more likely to be sampled according to their high similarity to the query. To further reduce CPU workload, MAGICPIG stores these tokens on GPU and does not apply LSH sampling to them. We leverage the recursive attention technique (Ye et al., 2024) to merge the attention output from CPU and GPU.

Our algorithm applies to a single attention head, see Algorithm 1. The details of **Encode**, **Query**, as well as the hash table construction, are described in prior work (Sadowski, 2007; Chen et al., 2020b). In Appendix E, we discuss the selection of LSH hyper-parameter ( $K$ ,  $L$ ).

## 5 Evaluation

In this section, we aim to demonstrate that MAGICPIG can speed up LLM decoding while preserving high accuracy. We first present MAGICPIG’s accuracy in downstream tasks, followed by our end-to-end system results showing wall-clock performance.

- In Section 5.1, we demonstrate MAGICPIG preserves high accuracy (less than 2% degradation) across moderate to long context tasks with computation cost 2% ~ 5% of full attention.
- In Section 5.2, we demonstrate the system performance of MAGICPIG, which achieves up to  $5\times$  throughput improvement and 54ms decoding latency on a single RTX 4090 for Llama-3.1-8B-Instruct with 96K context.
- In Section 5.3, we verify the effectiveness of centering, which is of vital importance for the success of sampling. Also, we demonstrate that MAGICPIG already outperforms TopK attention in the two aggregation tasks in Figure 1, indicating that sampling indeed goes beyond TopK attention.

### 5.1 MagicPIG Preserves Accuracy

We demonstrate that MAGICPIG can preserve accuracy in diverse tasks with less than 5% computation.

**Setup.** Our experiments are based on Llama (AI@Meta, 2024; Dubey et al., 2024; Touvron et al., 2023) models. Three types of tasks are included, which are 3 mid-context comprehensive tasks from lm-eval-harness (Gao et al., 2021) (GSM8K-CoT (Cobbe et al., 2021), MMLU-Flan-Cot-Fewshot (Hendrycks et al., 2020) and COQA (Reddy et al., 2019)), and 6 long context tasks from (Bai et al., 2023) (QASPER (Dasigi et al., 2021), LCC, Repobench-P (Liu et al., 2023), TriviaQA (Joshi et al., 2017), PRE and TREC (Li and Roth, 2002; Hovy et al., 2001)) and 13 synthetic tasks from RULER (Hsieh et al., 2024) (with 50 examples per task).

**Baselines.** Besides full attention, Quest (Tang et al., 2024) and its variants are used as baselines. In its default setting, Quest uses a “page size” of 16, i.e. 1/16 of the full attention cost. To compare the methods fairly in